

HERMÈS

Audit 2026

Recommandations stratégiques pour la sécurisation et l'industrialisation de la plateforme

Ce rapport présente les résultats de l'audit complet du codebase HERMÈS — application Next.js 16 dédiée à l'automatisation LinkedIn B2B. Il cartographie les risques par criticité, détaille les vulnérabilités identifiées sur les couches d'authentification, de données, de build et de sécurité applicative, et propose un plan de mise en œuvre priorisé sur douze semaines avec quick wins, efforts estimés et impact métier.

3

RISQUES P0

7

RISQUES P1

9

RISQUES P2

12

SEMAINES

CIBLE

Next.js 16 · React 19 · Prisma 6 · TypeScript 5

PÉRIMÈTRE

src/lib · src/app/api · prisma · middleware · next.config

DATE

29 juin 2026

AUTEUR

Audit Technique Interne

Table des matières

1. Synthèse exécutive	3
1.1 Contexte et objectifs de l'audit	3
1.2 Méthodologie	3
1.3 Synthèse des constats par criticité	3
2. Cartographie des risques	5
2.1 Vue d'ensemble par couche	5
2.2 Matrice criticité × probabilité	5
2.3 Score global de maturité	6
3. Recommandations P0 — Critiques	7
3.1 R-001 — Réécrire l'authentification	7
3.2 R-002 — Imposer le multi-tenant	8
3.3 R-003 — Désactiver ignoreBuildErrors	8
4. Recommandations P1 — Hautes	10
4.1 R-004 & R-005 — Base de données : aligner schéma et logs	10
4.2 R-007 — Rate-limit distribué	10
4.3 R-008 — Gestion d'erreurs API	11
4.4 R-009 — Stratégie de test	12
4.5 R-010 — Headers de sécurité	12
5. Recommandations P2 — Moyennes	14
5.1 R-011 — Éliminer les types any	14
5.2 R-012 — Activer l'optimisation d'images	14
5.3 R-013 — Optimiser ensureDefaultUser	14
5.4 R-014 — Consolider la configuration de déploiement	14
5.5 Améliorations transverses	15
6. Plan de mise en œuvre	16
6.1 Roadmap 12 semaines	16
6.2 Quick wins (semaine 1)	16
6.3 Effort cumulé par phase	17
7. Conclusion et prochaines étapes	18

7.1 Bilan 18

7.2 Prochaines étapes recommandées 18

7.3 Indicateurs de succès 18

1. Synthèse exécutive

1.1 Contexte et objectifs de l'audit

HERMÈS est une plateforme B2B d'automatisation LinkedIn construite sur Next.js 16, React 19, Prisma 6 et TypeScript 5. L'application orchestre la génération de contenu par IA, la publication programmée, la gestion de leads, les séquences email, les workflows métier, les webhooks, l'A/B testing et le suivi de conformité LinkedIn. Le codebase compte plus de trente routes API, une dizaine de moteurs métier et un schéma Prisma de près de 750 lignes.

Cet audit a été conduit dans une optique d'industrialisation : préparer la plateforme à une mise en production multi-tenant, à une certification RGPD et à une présentation à des investisseurs ou clients entreprise. L'objectif n'est pas de lister exhaustivement les défauts cosmétiques, mais d'identifier les risques bloquants — sécuritaires, architecturaux ou de dette technique — qui empêcheraient HERMÈS de passer à l'échelle supérieure. Chaque recommandation est priorisée, chiffrée en effort et associée à un impact métier mesurable.

1.2 Méthodologie

L'audit s'est appuyé sur cinq axes complémentaires. Premièrement, l'analyse statique du code source — lecture croisée du middleware, des configurations Next.js et Prisma, des routes API et des moteurs lib — a permis d'identifier les anti-patterns structurels. Deuxièmement, l'inspection du schéma de données Prisma a révélé des incohérences entre la configuration déclarée (PostgreSQL) et la configuration effective (SQLite). Troisièmement, la cartographie des dépendances et de la chaîne de build a mis en évidence des dérives de configuration. Quatrièmement, l'examen des mécanismes d'authentification, d'autorisation et de session a révélé des vulnérabilités critiques. Cinquièmement, l'évaluation de la stratégie de test a montré une absence totale de couverture automatisée.

1.3 Synthèse des constats par criticité

L'audit a identifié dix-neuf risques significatifs répartis en trois niveaux de priorité. Trois risques P0 — bloquants pour toute mise en production — concernent l'authentification, le modèle multi-tenant et la chaîne de build. Sept risques P1 doivent être traités dans les quatre à huit semaines pour atteindre un niveau de maturité acceptable. Neuf risques P2 relèvent de l'amélioration continue et peuvent être absorbés par l'équipe produit sur le trimestre.

3 Risques P0	7 Risques P1	9 Risques P2	19 Risques totaux
-----------------	-----------------	-----------------	----------------------

Verdict global

La plateforme HERMÈS démontre une excellente couverture fonctionnelle et une architecture modulaire solide. Cependant, dans son état actuel, elle n'est pas conforme aux exigences d'une mise en production multi-tenant : l'authentification repose sur un compte démo hardcodé, toutes les données sont partagées via un `DEFAULT_USER_ID` unique, et le build ignore silencieusement les erreurs TypeScript. Les corrections P0 représentent environ deux semaines de travail et débloquent la totalité de la roadmap d'industrialisation.

2. Cartographie des risques

2.1 Vue d'ensemble par couche

Les risques identifiés ont été classés selon cinq couches techniques distinctes. Cette segmentation permet d'orienter les actions correctives vers les équipes ou contributeurs concernés, et de visualiser les concentrations de dette technique. La couche sécurité concentre la majorité des risques critiques, suivie par la couche données qui souffre d'une incohérence de configuration entre développement et production.

Couche	P0	P1	P2	Total	Maturité
Sécurité & Auth	3	2	1	6	Faible
Données & Persistance	0	2	1	3	Moyenne
Build & TypeScript	1	1	0	2	Faible
API & Résilience	0	1	3	4	Moyenne
Observabilité & Tests	0	1	4	5	Très faible

Tableau 1 — Distribution des risques par couche technique et niveau de priorité.

2.2 Matrice criticité × probabilité

La matrice ci-dessous croise l'impact métier (sécurité, conformité, continuité de service, réputation) avec la probabilité d'occurrence observée. Les risques P0 se situent tous dans le quadrant « impact critique × probabilité élevée » — ils ne sont pas hypothétiques mais constituent des vulnérabilités actives qui seraient exploitées dès la première mise en ligne non protégée.

ID	Risque	Impact	Probabilité	Niveau
R-001	Authentification démo hardcodée	Critique	Élevée	P0
R-002	Pas de multi-tenant (DEFAULT_USER_ID)	Critique	Élevée	P0
R-003	ignoreBuildErrors masque les erreurs TS	Critique	Élevée	P0
R-004	Schéma PostgreSQL / SQLite incohérent	Élevé	Élevée	P1
R-005	Prisma log:query actif en production	Élevé	Élevée	P1
R-006	Aucune migration versionnée	Élevé	Moyenne	P1
R-007	Rate-limit in-memory (multi-instance)	Élevé	Élevée	P1
R-008	Routes API sans try/catch	Moyen	Élevée	P1

ID	Risque	Impact	Probabilité	Niveau
R-009	Absence de tests automatisés	Élevé	Certaine	P1
R-010	Headers de sécurité incomplets (CSP, HSTS)	Moyen	Élevée	P1
R-011	87 types any sur 30 fichiers	Moyen	Certaine	P2
R-012	images.unoptimized = true	Faible	Certaine	P2
R-013	ensureDefaultUser sur chaque écriture	Faible	Certaine	P2
R-014	Configuration de déploiement multiple	Faible	Moyenne	P2

Tableau 2 — Matrice des risques principaux avec impact, probabilité et niveau de priorité.

2.3 Score global de maturité

Sur une échelle de maturité à cinq niveaux — Initial, Reproductible, Défini, Maîtrisé, Optimisé — HERMÈS se positionne globalement au niveau « Reproductible » (niveau 2 sur 5). La couverture fonctionnelle et la structuration du code sont de qualité correcte, mais les pratiques de sécurité, de test et d'observabilité restent au niveau Initial. L'objectif post-audit est d'atteindre le niveau « Défini » sur toutes les couches sous trois mois, puis « Maîtrisé » sous six mois.

3. Recommandations P0 — Critiques

Engagement de délai

Les trois recommandations P0 doivent être traitées en moins de deux semaines ouvrées. Aucune mise en production, aucune démo client et aucun partage de preview public ne doivent avoir lieu avant leur résolution complète. Ces correctifs sont des prérequis à toute communication externe.

3.1 R-001 — Réécrire l'authentification

Constat

Le fichier `src/lib/auth-config.ts` implémente un unique compte démo hardcodé : `demo@hermes.app` avec le mot de passe `hermes2024`. La comparaison se fait en clair, sans hachage, sans salage, sans verrouillage anti-brute-force. Le secret JWT utilisé a une valeur de repli hardcodée — `"hermes-dev-secret-change-in-production"` — qui sera utilisée si `NEXTAUTH_SECRET` n'est pas défini en environnement, ce qui rend les tokens forgeables. Enfin, le middleware désactive complètement l'authentification en développement, créant un faux sentiment de sécurité lors des tests locaux.

Recommandation

Migrer vers un fournisseur OAuth réel (GitHub, Google) ou implémenter un provider Credentials avec stockage en base de données, mots de passe hachés via `argon2id` (préférable à `bcrypt` pour la résistance GPU), et vérification systématique en base. Ajouter un `rate-limit` dédié sur la route `/api/auth/callback/credentials` (5 tentatives / 15 minutes / IP). Supprimer le fallback de `NEXTAUTH_SECRET` : lancer une exception au démarrage si la variable n'est pas définie. Conserver l'authentification active en développement pour éviter les divergences de comportement entre environnements.

Effort et impact

Dimension	Valeur
Effort estimé	3 jours-homme
Complexité	Moyenne
Dépendances	Aucune
Impact sécurité	Critique
Impact conformité RGPD	Élevé

3.2 R-002 — Imposer le multi-tenant

Constat

Le fichier `src/lib/db.ts` exporte une constante `DEFAULT_USER_ID = "default"` utilisée par la quasi-totalité des routes API. Cela signifie que tous les leads, posts, métriques, contacts, deals, emails, workflows et notifications sont stockés sans isolation entre utilisateurs. Même si l'authentification était corrigée, n'importe quel utilisateur authentifié pourrait consulter et modifier les données de tous les autres utilisateurs. C'est un défaut architectural bloquant pour toute utilisation réelle en SaaS.

Recommandation

Remplacer toutes les références à `DEFAULT_USER_ID` par `session.user.id` obtenu via `getSessionSession()`. Ajouter un garde-fou centralisé — une fonction `requireUser()` qui lève une erreur 401 si la session est absente et renvoie l'identifiant utilisateur. Effectuer un audit systématique de chaque requête Prisma pour vérifier la présence d'un filtre `where: { userId }`. Compléter par des tests d'intrusion : un utilisateur A ne doit jamais pouvoir lire ou modifier une ressource appartenant à un utilisateur B.

Effort et impact

Dimension	Valeur
Effort estimé	5 jours-homme
Complexité	Élevée
Dépendances	R-001 (auth)
Impact sécurité	Critique
Impact business model	Critique — sans SaaS possible

3.3 R-003 — Désactiver `ignoreBuildErrors`

Constat

Le fichier `next.config.ts` contient la ligne `typescript: { ignoreBuildErrors: true }`. Cette configuration indique à Next.js de produire un bundle de production même si le compilateur TypeScript détecte des erreurs de types. En conséquence, des bugs silencieux — variables mal typées, propriétés inexistantes, nullabilités ignorées — peuvent passer en production sans qu'aucun signal d'alarme ne soit émis. L'audit a également relevé 87 occurrences de type `any` réparties sur 30 fichiers, ce qui suggère que le typage a été relâché pour faire taire le compilateur plutôt que pour refléter une intention.

Recommandation

Supprimer immédiatement la ligne `ignoreBuildErrors`. Lancer `npx tsc --noEmit` pour obtenir la liste exhaustive des erreurs. Les corriger par ordre de criticité : d'abord les erreurs dans les routes API et les moteurs lib (impact runtime), ensuite les erreurs dans les composants UI (impact visuel). Pour les `any` légitimes (rare), utiliser `unknown` avec un narrowing explicite ou créer un type dédié. Ajouter `tsc --noEmit` comme étape obligatoire dans la pipeline CI.

Effort et impact

Dimension	Valeur
Effort estimé	4 jours-homme
Complexité	Faible
Dépendances	Aucune
Impact qualité	Élevé
Impact maintenance	Élevé — diminue la dette future

4. Recommandations P1 — Hautes

Les recommandations P1 ne sont pas bloquantes pour une mise en ligne interne ou une démo encadrée, mais doivent être traitées avant toute communication publique, présentation client ou montage en charge. Elles concernent la cohérence de la base de données, la résilience applicative, la sécurité des en-têtes HTTP et la stratégie de test.

4.1 R-004 & R-005 — Base de données : aligner schéma et logs

Constat

Le schéma Prisma déclare `provider = "postgresql"` mais le fichier `.env` utilise `DATABASE_URL=file:...sqlite`. Cette incohérence signifie que certaines fonctionnalités Prisma (types spécifiques PostgreSQL, migrations avancées, JSON indexé) pourraient échouer en production. Par ailleurs, le client Prisma est instancié avec `log: ['query']`, ce qui produit une ligne de log par requête SQL — potentiellement des milliers par minute en production. Cette verbosité saturera les agrégateurs de logs (Datadog, ELK), masquera les vraies erreurs et augmentera significativement les coûts d'observabilité.

Recommandation

Aligner le schéma et la configuration runtime : soit conserver SQLite en modifiant le provider du schéma à `sqlite`, soit migrer réellement vers PostgreSQL (recommandé pour la production). Dans les deux cas, versionner les migrations via `prisma migrate dev` plutôt que `prisma db push`. Concernant les logs, passer à `log: process.env.NODE_ENV === 'development' ? ['query', 'error', 'warn'] : ['error', 'warn']`. Enfin, remplacer la fonction `ensureDefaultUser()` appelée à chaque écriture par un seed unique exécuté au déploiement.

Sous-tâche	Effort	Priorité
Aligner provider Prisma ↔ DATABASE_URL	2 heures	P1
Activer prisma migrate (première migration)	4 heures	P1
Désactiver log:query en production	30 minutes	P1
Supprimer ensureDefaultUser des routes write	1 jour	P2

4.2 R-007 — Rate-limit distribué

Constat

Le middleware utilise un `Map` en mémoire pour suivre le comptage de requêtes par adresse IP. Cette approche fonctionne sur une instance unique mais devient inopérante dès que l'application est déployée sur plusieurs

instances (Render, Netlify Functions, conteneurs Kubernetes). Chaque instance ayant sa propre Map, un attaquant peut envoyer $N \times 60$ requêtes par minute en répartissant sa charge sur plusieurs instances via le load balancer. Le seuil de 60 requêtes par minute est par ailleurs trop généreux pour les routes d'authentification et trop strict pour les routes de lecture de données agrégées.

Recommandation

Migrer vers un store distribué — Upstash Redis (serverless, gratuit jusqu'à 10 000 commandes par jour) ou un Redis managé. Utiliser la bibliothèque `@upstash/ratelimit` qui implémente l'algorithme sliding-window. Définir des limites par catégorie de route : auth (5 req/min), écriture (30 req/min), lecture (120 req/min), AI (10 req/min pour limiter les coûts). Ajouter une header `X-RateLimit-Remaining` pour permettre aux clients d'adapter leur cadence.

Catégorie de route	Limite	Window	Store
/api/auth/*	5	15 min	Redis
/api/data/* (POST/PUT/DELETE)	30	1 min	Redis
/api/data/* (GET)	120	1 min	Redis
/api/ai/*	10	1 min	Redis
/api/linkedin/*	20	1 min	Redis

4.3 R-008 — Gestion d'erreurs API

Constat

L'audit a relevé que de nombreuses routes API — par exemple `src/app/api/data/leads/route.ts` — n'emballent pas leurs appels Prisma dans un bloc `try/catch`. En cas d'erreur (contrainte unique violée, record introuvable, timeout de base de données), Next.js renvoie une réponse HTTP 500 générique avec le message d'erreur Prisma en clair — ce qui peut divulguer des informations sur le schéma (noms de colonnes, relations) à un attaquant. L'absence de format d'erreur structuré complique également la vie des consommateurs frontend qui ne peuvent pas distinguer une erreur client (400) d'une erreur serveur (500).

Recommandation

Créer un wrapper `withErrorHandler(handler)` qui emballe systématiquement le handler de chaque route. Ce wrapper capture les erreurs Prisma connues (P2002 unique constraint → 409, P2025 record not found → 404, P1001 timeout → 503) et renvoie un format JSON normalisé `{ error: { code, message, details? } }`. Logger les erreurs 500 via le logger structuré avec un identifiant de corrélation. Ne jamais renvoyer le message Prisma brut en production. Migrer les routes existantes une par une (une route = un commit pour faciliter la review).

4.4 R-009 — Stratégie de test

Constat

Le projet ne contient aucun test automatisé : zéro fichier `.test.ts`, aucun framework de test installé (pas de Vitest, Jest, Playwright ou Testing Library), aucune commande de test dans `package.json`. Cette absence de couverture rend chaque refactor risqué : une régression peut être introduite sans être détectée jusqu'à la production. Elle bloque également toute contribution externe — un développeur ne pouvant pas vérifier que son changement ne casse rien, la revue de code devient le seul filet de sécurité, ce qui ne scale pas.

Recommandation

Adopter une stratégie de test en trois couches complémentaires. Couche unitaire avec Vitest pour les fonctions pures (`format.ts`, `utils.ts`, moteurs lib isolés via mock Prisma). Couche d'intégration avec Vitest + une base de test SQLite en mémoire pour valider les routes API complètes. Couche end-to-end avec Playwright pour les parcours critiques (login, création de lead, génération de post, programmation). Cible initiale : 60 % de couverture sur `src/lib`, 40 % sur `src/app/api`, 5 parcours E2E. Ajouter une étape CI GitHub Actions qui lance les tests sur chaque pull request et bloque la fusion si la couverture baisse.

Couche	Outil	Cible couverture	Délai
Unitaire	Vitest	60% sur lib	S2
Intégration	Vitest + SQLite	40% sur API	S3
E2E	Playwright	5 parcours clés	S4
CI	GitHub Actions	100% PRs	S2

4.5 R-010 — Headers de sécurité

Constat

Le middleware ajoute trois en-têtes : `X-Frame-Options: DENY`, `X-Content-Type-Options: nosniff`, et `Referrer-Policy: strict-origin-when-cross-origin`. C'est un minimum viable, mais il manque quatre en-têtes critiques : `Content-Security-Policy` qui prévient les attaques XSS, `Strict-Transport-Security` qui force HTTPS, `Permissions-Policy` qui limite l'utilisation des API navigateur (caméra, micro, géoloc), et `X-DNS-Prefetch-Control` qui contrôle le pré-resolve DNS.

Recommandation

Ajouter les quatre en-têtes manquants dans le middleware. Pour la CSP, commencer en mode `report-only` pendant deux semaines pour collecter les violations sans casser l'application, puis basculer en

mode bloquant. Schéma recommandé : default-src 'self'; script-src 'self' 'unsafe-inline'; style-src 'self' 'unsafe-inline'; img-src 'self' data: https:; connect-src 'self' https://api.linkedin.com https://googleapis.com; font-src 'self' https://fonts.gstatic.com; frame-ancestors 'none'. HSTS: max-age=63072000; includeSubDomains; preload. Permissions-Policy : désactiver caméra, micro, géoloc, payment.

5. Recommandations P2 — Moyennes

Les recommandations P2 relèvent de l'amélioration continue. Elles ne sont pas bloquantes mais contribuent significativement à la maintenabilité, la performance et l'expérience développeur. Elles peuvent être absorbées par l'équipe produit au fil de l'eau, idéalement en réservant 20 % du temps de chaque sprint à la dette technique.

5.1 R-011 — Éliminer les types `any`

Le codebase contient 87 occurrences du type `any` réparties sur 30 fichiers, concentrées principalement dans les routes API (`export/route.ts` en compte 19 à lui seul) et dans les moteurs `notifications`, `feedback` et `workflow`. Le type `any` désactive la vérification de types, ce qui masque les bugs potentiels et empêche l'IDE de proposer des autocomplétions pertinentes. La strategy : remplacer chaque `any` par un type précis (interface, type alias, ou `unknown` avec narrowing), un fichier à la fois, en s'aidant du compilateur pour détecter les régressions. L'objectif réaliste est de passer sous 20 occurrences en un trimestre.

5.2 R-012 — Activer l'optimisation d'images

La configuration `images.unoptimized = true` désactive le service d'optimisation d'images de Next.js. Conséquence : chaque image envoyée au navigateur l'est dans sa résolution et son format d'origine. Sur une page dashboard contenant des screenshots, des avatars et des illustrations, cela peut représenter plusieurs mégaoctets transférés inutilement. Pour réactiver : supprimer la ligne `unoptimized` et configurer `images.remotePatterns` avec la liste blanche des domaines externes (LinkedIn, Google avatars, etc.). Si l'hébergeur ne supporte pas l'optimisation serverless (Netlify), utiliser le loader `imgix` ou `cloudinary`.

5.3 R-013 — Optimiser `ensureDefaultUser`

La fonction `ensureDefaultUser()` est appelée avant chaque opération d'écriture dans les routes API. Elle exécute un `findUnique` puis potentiellement un `create` sur la table User. Cela ajoute un round-trip base de données systématique — environ 5 à 15 ms par requête — ce qui dégrade le p99 latence et consomme inutilement des connexions. La correction : exécuter cette fonction une seule fois au démarrage de l'application (dans `instrumentation.ts` fourni par Next.js) ou via un script de seed exécuté au déploiement. À terme, cette fonction doit disparaître complètement une fois le multi-tenant réel en place (R-002).

5.4 R-014 — Consolider la configuration de déploiement

Le dépôt contient cinq fichiers de configuration de déploiement distincts : `render.yaml`, `netlify.toml`, `docker-compose.yml`, `build.sh`, `Caddyfile`. Cette pluralité crée une confusion sur la cible de production officielle et multiplie les risques de divergence de configuration. Recommandation : choisir une cible unique (Render pour la simplicité, ou Docker + Caddy pour la maîtrise), supprimer les fichiers obsolètes, et documenter le choix retenu dans un fichier `DEPLOYMENT.md` à la racine du dépôt.

5.5 Améliorations transverses

Au-delà des risques identifiés, plusieurs améliorations transverses pourraient être conduites en parallèle sans attendre. Mise en place d'un `pre-commit` avec husky + lint-staged pour exécuter ESLint et Prettier sur les fichiers modifiés. Adoption d'une convention de commit (Conventional Commits) pour faciliter la génération automatique de changelogs. Documentation des décisions architecturales via ADR (Architecture Decision Records) dans un dossier `docs/adr/`. Mise en place d'un tableau de bord d'observabilité (Grafana Cloud gratuit) exposant les métriques clés : latence p50/p95/p99 des routes API, taux d'erreur, débit, taille de la base.

6. Plan de mise en œuvre

6.1 Roadmap 12 semaines

La roadmap ci-dessous séquence les recommandations sur douze semaines en quatre phases. Chaque phase produit un livrable démontrable qui peut être présenté en revue de direction. Les efforts sont estimés en jours-homme sur la base d'un développeur senior connaissant le codebase ; un développeur junior devra multiplier par 1,5 à 2.

Semaine	Phase	Activité	Effort (jh)	Livrable
S1-S2	Phase 1 — P0	Auth réelle + multi-tenant	8	PR auth + tests isol.
S3	Phase 1 — P0	Désactiver ignoreBuildErrors + fix TS	4	Build propre
S4	Phase 2 — DB	Aligner provider + migrations	1	Schéma cohérent
S4	Phase 2 — DB	Désactiver log:query en prod	0.5	Logs propres
S5-S6	Phase 2 — Sécurité	Rate-limit Redis + headers CSP/HSTS	4	Middleware renforcé
S6-S7	Phase 2 — Résilience	withErrorHandler + migration routes	5	API résiliente
S7-S9	Phase 3 — Tests	Vitest setup + tests lib	8	Couverture 60% lib
S9-S10	Phase 3 — Tests	Tests API + 5 parcours E2E	6	CI bloquante
S10-S12	Phase 4 — Dette	Réduction des any + optimisation images	5	Code plus propre
S12	Phase 4 — Dette	Consolidation config déploiement	2	DEPLOYMENT.md

Tableau 3 — Roadmap séquentielle sur 12 semaines avec livrables intermédiaires.

6.2 Quick wins (semaine 1)

Trois actions peuvent être menées dès la première semaine avec un effort minimal et un effet immédiat sur la sécurité et la qualité. Ces quick wins ne nécessitent aucune dépendance externe et peuvent être livrés en un seul commit chacun.

- **Désactiver ignoreBuildErrors** et corriger les 5 erreurs TypeScript les plus bloquantes — 4 heures, impact qualité immédiat.
- **Ajouter les en-têtes CSP, HSTS, Permissions-Policy** dans le middleware en mode report-only — 1 heure, impact sécurité élevé.

- **Supprimer le fallback NEXTAUTH_SECRET** et lancer une exception au démarrage si absent — 30 minutes, impact sécurité critique.

6.3 Effort cumulé par phase

Phase	Période	Effort (jh)	% total	Effet
Phase 1 — P0	S1-S3	12	27%	Sécurité bloquante
Phase 2 — DB/Sécu/Résilience	S4-S7	10.5	24%	Industrialisation
Phase 3 — Tests	S7-S10	14	32%	Régression maîtrisée
Phase 4 — Dette	S10-S12	7	16%	Maintenabilité
Total	S1-S12	43.5	100%	Niveau « Défini »

Tableau 4 — Répartition de l'effort total estimé à 43,5 jours-homme.

7. Conclusion et prochaines étapes

7.1 Bilan

HERMÈS est une plateforme techniquement ambitieuse qui démontre une excellente compréhension des besoins métier de l'automatisation LinkedIn B2B. La richesse fonctionnelle — orchestrateur d'agents, workflow engine, A/B testing, conformité, CRM intégré — est remarquable pour un projet de cette envergure. L'architecture modulaire, avec neuf moteurs lib indépendants et persistance Prisma, est saine et facile à étendre. La récente migration des stores in-memory vers la base de données (BUG-H2) témoigne d'une capacité d'amélioration continue déjà à l'œuvre.

Cependant, le projet porte encore les stigmates d'un développement itératif rapide : l'authentification démo n'a jamais été remplacée, le multi-tenant a été différé, et la chaîne de build a été désamorcée pour avancer plus vite. Ces raccourcis étaient légitimes en phase d'exploration, mais deviennent des blockeurs dès lors que l'on envisage une mise en production ou une démonstration à des clients entreprise. L'audit a identifié trois risques P0 qui représentent environ deux semaines de travail — un investissement modeste au regard du déblocage de toute la roadmap d'industrialisation.

7.2 Prochaines étapes recommandées

La séquence d'action recommandée est la suivante. Premièrement, valider publiquement ce rapport avec l'équipe technique et arbitrer les priorités en fonction du contexte business (levée, signature client, date de lancement). Deuxièmement, planifier un sprint P0 dédié de deux semaines sans aucune autre tâche produit pour absorber les correctifs R-001, R-002 et R-003. Troisièmement, mettre en place la pipeline CI (GitHub Actions) avec `tsc --noEmit`, ESLint et les premiers tests Vitest dès la fin du sprint P0. Quatrièmement, conduire un audit de pénétration externe après la phase 2 pour valider indépendamment la surface d'attaque. Cinquièmement, planifier un second audit interne dans six mois pour mesurer la progression vers le niveau de maturité « Maîtrisé ».

7.3 Indicateurs de succès

Pour mesurer l'efficacité du plan de mise en œuvre, trois indicateurs clés sont recommandés. Le taux de couverture de tests doit passer de 0 % à 60 % sur `src/lib` et 40 % sur `src/app/api` d'ici la fin de la phase 3. Le nombre de vulnérabilités critiques détectées par un scan automatisé (Snyk, Dependabot, OWASP ZAP) doit tomber à zéro après la phase 2. Le temps de build en CI doit rester sous 5 minutes pour préserver la fluidité de développement. Ces indicateurs doivent être publiés dans un dashboard interne et revus en revue de direction mensuelle.

Recommandation finale

L'investissement total pour atteindre le niveau de maturité « Défini » est estimé à 43,5 jours-homme répartis sur 12 semaines, soit approximativement un développeur senior à mi-temps. C'est un investissement raisonnable au regard de l'enjeu : transformer HERMÈS d'un prototype avancé en une plateforme SaaS entreprise prêt à accueillir ses premiers clients payants. Le passage à l'échelle supérieure ne se fera pas sans ces fondations solides.